Freight rate prediction

Machine Learning Engineer assessment - Methodology and results

1. Goal and data

The goal is to predict posted_rate in dollars for all 12,000 loads in the provided final prediction dataset and submit the fixed December prediction chart. The labeled development dataset includes 48,000 loads from Jan 1 - Oct 31, 2025. The final data includes dates Nov 1 - Dec 31, 2025 and has no actual prices.

There is a separate data file with 31 days of December rows for one fixed shipment. Predictions should be made on these using the same final fitted pipeline that will be used for the 12,000 load submission. Provided score.py checks the output files and generates the chart.

2. Data exploration and preprocessing

Finding	Treatment
300 missing weights, 292 non positive weights	Non positive weights are converted to missing, resulting in 592 missing weights, training-partition median imputation is applied in the pipeline
374 missing market index values	market_index is excluded from the selected feature set, its predictive value was not evaluated
Unique development Load IDs	IDs are used for output matching only, they are excluded from model features
Unusually expensive loads	Retained in training and evaluation because there is insufficient evidence that they are wrong

Development distances range from 70 to 3,439.8 miles. After invalid weights are replaced, observed weights range from 5,000 to 47,500 lb. All observed distances and posted rates are positive. The median posted rate is $2,030.76, the 99th percentile is $5,972.83, and the maximum is $25,533.00. The selected model uses distance, weight, pickup city, delivery city and equipment type. Numeric imputation includes missing-value indicators. Pickup, delivery and equipment are one-hot encoded with unknown-category handling. Pipeline preprocessing is fitted exclusively on each training partition, preventing evaluation rows from determining medians or categorical vocabularies. Coordinates, market_index and quote_signal are not included. Their predictive value was not tested in this implementation. No rows are removed solely because their price is extreme

3. Chronological validation design

A chronological split matches the future-prediction task: learn from earlier loads and evaluate on later loads. A random split would mix time periods and could give an optimistic picture if pricing relationships change. All observations from a given date remain in one partition.

Partition	Dates	Loads	Role

Training	Jan-Aug 2025	38477	Fit candidate models
Development	September 2025	4670	Compare models and select settings
Evaluation	October 2025	4853	Evaluate after fitting Jan-Sep

The baseline uses the January-August median posted rate ($2,026.04) as a constant prediction for every September load. MAE, the mean absolute dollar difference between predicted and actual prices, is the primary comparison metric. A median is appropriate for this baseline because it minimizes training absolute error among constant predictions. September MAE is used for model and parameter selection. The chosen model is then trained on January-September and evaluated on October. After evaluation, a fresh clone of the selected pipeline is fitted on all 48,000 labeled loads to predict the final November-December inputs

4. Model experiments and selection

Candidate	September MAE
Training-median baseline	$1,151.11
Random forest, 100 trees	$154.92
Original gradient boosting, without dates	$116.41
Tuned gradient boosting, without dates	$110.87
Tuned gradient boosting, with dates	$118.42

Random forest provides a substantial improvement over the constant baseline. Gradient boosting with absolute-error loss improves further. This loss aligns with the selected MAE metric and limits the fitting influence of extreme target values compared with squared-error loss. The date experiment adds day of week, a weekend indicator and annual sine/cosine features. For the tuned model these increase September MAE from $110.87 to $118.42, so the final model excludes dates. This result applies to the tested representation and model; it does not establish that calendar information is universally unhelpful. The final model uses 200 estimators, learning rate 0.05, maximum depth 4, minimum samples per leaf 20, absolute-error loss and random seed 42.

5. Parameter comparison and october results

Trees	Learning rate	Depth	Min leaf size	September MAE
100	0.10	3	1	$116.41
200	0.05	3	1	$115.09
200	0.05	4	1	$111.12

200	0.05	4	20	$110.87

Settings are selected by the lowest September MAE. The winning model reduces development error by approximately 4.8% relative to the original boosting model and 90.4% relative to the median baseline. The $0.25 difference between the two depth-4 candidates is small and should not be interpreted as strong evidence of a general advantage.

Model	Evaluation period	MAE
Original Boosting	Oct 2025	$121.18
Tuned Boosting	Oct 2025	$115.76

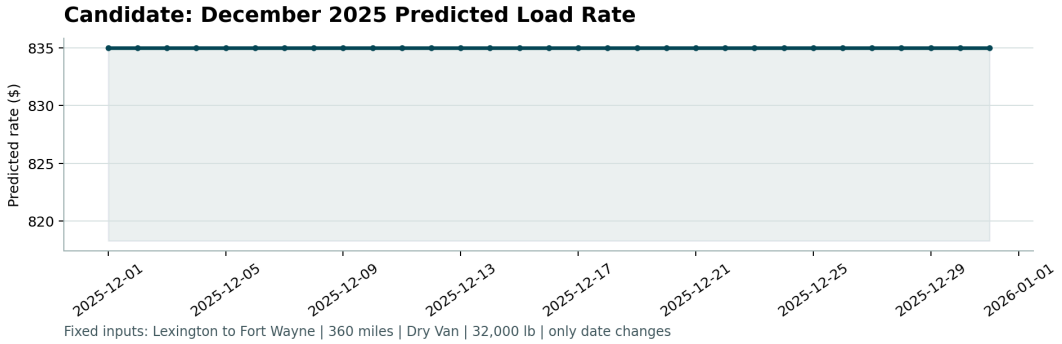
For each October evaluation, the model is fitted on January-September only. The tuned model has October MAE of $115.76, compared with its September MAE of $110.87. These similar averages provide evidence of performance across two successive months, but do not measure hidden November-December accuracy. Evaluation history: October was initially inspected using the original model before additional tuning. Later parameter selection used September MAE, but October is therefore a previously inspected evaluation period rather than a completely untouched final holdout. The original October score is not attributed to the tuned model.

6. Error analysis and limitations

September error analysis of the random forest showed median absolute error of $48.65, a 95th-percentile error of $399.30, and a maximum error of $16,258.90. Its largest errors were underpredictions of unusually expensive loads. These figures describe the random forest experiment, not the selected boosting model. Large target values remain in every evaluation. An average MAE does not imply that every load is predicted within that dollar amount. Further investigation would be needed to determine whether expensive loads reflect valid business conditions or data-quality issues. The labeled data covers only ten months of one year. Reliable repeated winter or holiday patterns cannot be established. Repeated selection on September can overfit decisions to that month, and October had already been inspected. Additional temporal folds and evaluation of the omitted signals would strengthen a future iteration; these were not part of the completed implementation.

7. Required December prediction chart

The 31 rows cover December 1-31, 2025. Each uses Lexington to Fort Wayne, 360 miles, Dry Van and 32,000 lb. The only changing input is date. The selected model excludes dates, so all rows receive the same predicted rate: $834.97 (unrounded value approximately $834.965242)



8. Reproducibility and final output checks

The implementation is in explore1.ipynb and uses pandas, NumPy, scikit-learn 1.6.1 and matplotlib. Open the project folder in VS Code, select the intended Python notebook kernel, install requirements.txt in that environment, restart the kernel and run all cells in order. The notebook resolves input paths relative to its working directory. Final predictions are matched to the template by load_id rather than assuming identical row order. The output data/validation_predictions.csv contains exactly load_id,predicted_rate. The completed December file retains its original seven-column schema. The supplied scorer confirmed 12,000 valid final predictions and 31 valid December predictions, and generated scorer_results/candidate_december.png.

From the project folder, run:

```
python score.py --predictions
data/validation_predictions.csv --december-predictions
data/december-chart-inputs.csv
```

The submission consists of the solution repository with dependencies and run instructions, the 12,000-row prediction CSV, this report with the required chart, and a 2-3 minute Loom walkthrough. Passing the scorer confirms the output contract; final prediction accuracy remains subject to the employer's hidden evaluation.